Class Activity: JSONB and Vector Databases

Members: Seth Garciano, Tristin Bates, Troy Franks, Yisong Wang

Database Implementation

Component	Actual result
Database	ca3_streaming, existing local PostgreSQL service on port 5432
Versions	PostgreSQL 18.6; pgvector 0.8.7
Catalogue	60 title embeddings and 5 phrase embeddings; 384 dimensions each
Synthetic sample	24 users; 144 events (72 play, 24 pause, 24 rate, 24 search)
Validation	Six queries executed; four invalid inserts rejected; no orphan events or missing genre arrays
Captured at	2026-11-03 18:31:23 UTC

Purpose and Scope

Our design combines relational identifiers and foreign keys with JSONB fields that have varying attributes. The supplied sentence embeddings support ranking titles by meaning, while normal SQL applies activity-based filters.

Design Decisions

Table	Purpose and key choices
users	user_id primary key; unique email; country and subscription plan as structured fields. preferences JSONB is an object with a required favourite_genres array.
activity_logs	activity_id primary key; user_id references users; title_id references titles. A search event has no title, while play, pause and rate events require one. occurred_at is timestamptz.
titles	Instructor catalogue schema with text description and vector(384). Title IDs link activity to the catalogue.
search_phrases	Independent phrase IDs, phrase text and vector(384). The phrase vectors are compared with title vectors; there is no foreign key relationship.

Why These Fields Are JSONB

Users share a genre array and subtitle language, but only some profiles contain autoplay or nested parental controls. Keeping these optional settings in JSONB allows for another preference key without adding a nullable column to every profile. Identity, email, plan, and relationships remain structured.

Events use different payloads:

- plays store minutes_watched and device
- pauses store paused_at_minute and reason
- ratings store stars
- searches store query and results_count

A shared event table keeps one timeline while JSONB holds event-specific fields.

Constraints and Indexes

CHECK constraints enforce JSON objects, the required genre array, four supported event types and typed event details. Ratings must be integers from 1 to 5. Two GIN indexes cover preferences and event_data. B-tree indexes support user timelines and title joins. The supplied titles table has a cosine HNSW index.

Query 1: Users who like Mystery

```
SELECT user_id, full_name, plan,
        preferences -> 'favourite_genres' AS favourite_genres
 FROM users
 WHERE preferences @> '{"favourite_genres":["Mystery"]}'::jsonb
 ORDER BY user_id;
```

Actual PostgreSQL output

User ID	Name	Plan	Favourite genres
1	Alex Morgan	Basic	Mystery, Sci-fi
8	Avery Singh	Standard	Reality, Mystery
9	Quinn Martin	Premium	Mystery, Sci-fi
16	Finley Adams	Basic	Reality, Mystery
17	Harper Evans	Standard	Mystery, Sci-fi
24	Payton Ward	Premium	Reality, Mystery

The @> operator finds profiles whose genre array contains Mystery, even when the object has other keys. Six users match: IDs 1, 8, 9, 16, 17 and 24.

Query 2: Top ten titles by recorded viewing minutes

```
SELECT t.title_id, t.title,
        sum((a.event_data ->> 'minutes_watched')::numeric) AS total_minutes,
        count(*) AS play_events
 FROM activity_logs a
 JOIN titles t ON t.title_id = a.title_id
 WHERE a.event_data @> '{"event_type":"play"}'::jsonb
 GROUP BY t.title_id, t.title
 ORDER BY total_minutes DESC, t.title_id
 LIMIT 10;
```

Actual PostgreSQL output

Title ID	Title	Minutes	Plays
11	Fourth Quarter	116	2
8	The Glass Orchard	112	2
5	Under the Canopy	108	2
12	Iron Meridian	108	2
9	Hollow Pines	106	2
2	Orbit of Ash	104	2
6	Midnight Ledger	104	2
3	Crumb & Clue	102	2
59	After Dark	70	1
10	Starward Academy	68	2

The ->> operator extracts minutes as text, so the query casts it to numeric before summing play events. Fourth Quarter ranks first with 116 recorded minutes from two plays; these are synthetic viewing totals, not unique viewers or a popularity estimate.

Query 3: Favourite genres across users

```
SELECT g.genre, count(*) AS users_with_genre
 FROM users u
 CROSS JOIN LATERAL jsonb_array_elements_text(
     u.preferences -> 'favourite_genres'
 ) AS g(genre)
 GROUP BY g.genre
 ORDER BY users_with_genre DESC, g.genre;
```

Actual PostgreSQL output

Genre	Users
Animation	6
Comedy	6
Documentary	6
Drama	6
Horror	6
Mystery	6
Reality	6
Sci-fi	6

jsonb_array_elements_text expands each favourite_genres array into one row per entry. Each of the eight genres appears for six users in this balanced sample; each profile has distinct genre entries, so count(*) equals the user count here.

Source: results/03_jsonb_array.csv. Values copied from the captured database execution; row order preserved.

Query 4: Five matches for something scary in the woods

```
SELECT t.title_id, t.title, t.genre,
       round((t.embedding <=> p.embedding)::numeric, 3) AS cosine_distance
 FROM titles t
 CROSS JOIN search_phrases p
 WHERE p.phrase_id = 4
 ORDER BY t.embedding <=> p.embedding, t.title_id
 LIMIT 5;
```

Actual PostgreSQL output

Title ID	Title	Genre	Distance
56	The Woods Remember	Horror	0.602
59	After Dark	Documentary	0.638
25	Little Fox Detectives	Animation	0.648
51	The Missing Hour	Mystery	0.666
31	Wild Kitchen	Food & Travel	0.671

Phrase 4 is "something scary in the woods". The Woods Remember has the lowest cosine distance (0.602), while the mixed-genre results show that semantic similarity is not a strict horror-genre filter.

Query 5: Five titles like Crumb & Clue

```
SELECT t.title_id, t.title, t.genre,
       round((t.embedding <=> seed.embedding)::numeric, 3) AS cosine_distance
 FROM titles t
 CROSS JOIN titles seed
 WHERE seed.title_id = 3 AND t.title_id <> seed.title_id
 ORDER BY t.embedding <=> seed.embedding, t.title_id
 LIMIT 5;
```

Actual PostgreSQL output

Title ID	Title	Genre	Distance
43	Small Town Spells	Fantasy	0.572
14	Thin Blue Lies	Crime	0.629
17	Signal Lost	Thriller	0.670
48	The Cartographer's Daughter	Historical Drama	0.677
39	Northern Lights Diner	Drama	0.680

Title 3, Crumb & Clue, is the seed and is excluded from its own recommendations. Small Town Spells is the nearest remaining title at 0.572; the distance measures embedding proximity, not a probability that a viewer will like it.

Query 6: Recommendations for user 1 excluding watched titles

```
WITH last_watched AS (
     SELECT title_id
     FROM activity_logs
     WHERE user_id = 1
       AND event_data @> '{"event_type":"play"}'::jsonb
     ORDER BY occurred_at DESC, activity_id DESC
     LIMIT 1
 )
 SELECT t.title_id, t.title, t.genre,
        round((t.embedding <=> seed.embedding)::numeric, 3) AS cosine_distance
 FROM last_watched lw
 JOIN titles seed ON seed.title_id = lw.title_id
 CROSS JOIN titles t
 WHERE NOT EXISTS (
     SELECT 1
     FROM activity_logs watched
     WHERE watched.user_id = 1
       AND watched.title_id = t.title_id
       AND watched.event_data @> '{"event_type":"play"}'::jsonb
 )
 ORDER BY t.embedding <=> seed.embedding, t.title_id
 LIMIT 5;
```

Actual PostgreSQL output

Title ID	Title	Genre	Distance
43	Small Town Spells	Fantasy	0.572
14	Thin Blue Lies	Crime	0.629
17	Signal Lost	Thriller	0.670
48	The Cartographer's Daughter	Historical Drama	0.677
39	Northern Lights Diner	Drama	0.680

The latest play for user 1 is title 3, and NOT EXISTS removes all three previously played titles (1, 2 and 3). This run returns the same five rows as Query 5 because titles 1 and 2 were not in that top five, and the seed was already excluded there.

Validation and Scalability

The temporary vector test returned IDs 1, 3, and 2, with cosine distances of 0.0000, 0.0085, and 0.2857. All 60 title vectors and five phrase vectors contain 384 values each. The database also has no orphan events or missing favourite_genres arrays. All six query results were checked against the original saved data, including the cosine rankings of the generated title vectors.

JSONB stores data in a format that PostgreSQL can search and index. GIN indexes can help with certain JSONB searches, such as checking whether specific values exist. However, they do not make every JSONB operation faster. In our tests, PostgreSQL used sequential scans because the tables were small.

For the vector searches, catalogue items and search phrases use the same model and vector size. HNSW indexes are designed to make similarity searches faster, but there is a tradeoff between speed, memory use, and accuracy. In our tests, the queries compared the title vectors directly instead of using the HNSW index. This means the top five results could be checked against a full comparison. These results show that the queries work, but they do not measure how well HNSW performs on larger datasets.

One benefit of using JSONB is that we can add optional preferences or extra details to existing events without needing to change the database tables. However, adding a completely new event type would still require updating the known_event_type and valid_event_details checks. This means JSONB gives us some flexibility, but we still need to keep the data structure consistent.

If the database grows, we may need to look at splitting activity data into time-based partitions, setting up data retention rules, and adding indexes for commonly searched JSONB fields. Vector searches may also need further testing to make sure results remain accurate when filters are applied. These decisions would depend on how the database performs with more realistic amounts of data.

One current limitation is that the favourite_genres constraint only checks whether the array exists. It does not check whether the genre names or the types of values inside the array are valid.

Contribution Table

Member	Contribution
Seth Garciano	Did queries 1 – 3 and verified them
Tristin Bates	Prepared the document and write report
Troy Franks	Did queries 4 – 6
Yisong Wang	Create schema diagrams and GitHub submission